



Patrón arquitectónico: MVVM

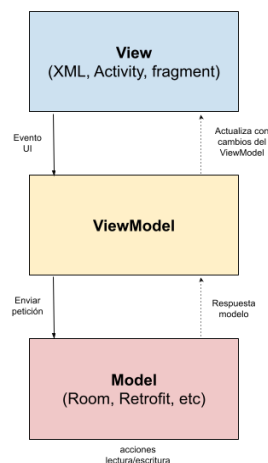
Una arquitectura no es más que una forma de organizar el código para poder llevar un control sobre nuestra aplicación y facilita el mantenimiento tal y como nuestra aplicación crece se vuelve más complejo.

MVVM significa Model-View-ViewModel y es un patrón de arquitectura para organizar código en aplicaciones, especialmente en Android. El modelo-vista-modelo de vista es una arquitectura que se ha hecho muy popular desde que Google la convirtiera en su arquitectura oficial al lanzar la [guía de arquitecturas](#).

Su objetivo principal es la de separar la lógica de la interfaz de usuario, para que la app sea más limpia, fácil de mantener y escalable.

- Model. Representa los datos y la lógica de negocio. Tendríamos data classes y repositorios
- View. Es la interfaz de usuario. Actividades, fragmentos y los xml (layouts, strings...)
- ViewModel. Es el puente/conexión entre las Views y los Models

Por tanto: La View solicita datos (o simplemente observa el estado), el ViewModel obtiene esos datos del Model o del Repository y cuando los datos cambian, el ViewModel notifica a la View automáticamente mediante LiveData.



Patrón del observador LiveData



LiveData es un componente de Android que actúa como un contenedor de datos observable. Esto significa que otros componentes, como Actividades o Fragmentos, pueden observar los cambios en los datos contenidos en LiveData.

Lo más importante de LiveData es que es consciente del ciclo de vida de sus observadores. Esto implica que solo actualiza los componentes que se encuentran en un estado de ciclo de vida activo, es decir, STARTED o RESUMED. Los observadores que están en un estado inactivo (por ejemplo, CREATED o DESTROYED) no reciben notificaciones sobre los cambios de datos.

Para utilizar LiveData, se registra un observador asociado a un objeto que implemente la interfaz LifecycleOwner (como una Actividad o Fragmento). Esta asociación permite que LiveData gestione automáticamente la suscripción del observador y lo elimine cuando su ciclo de vida llega al estado DESTROYED, evitando así fugas de memoria.

En resumen, LiveData:

- Es un contenedor de datos observable.
- Actualiza solo a observadores activos (STARTED o RESUMED).
- Comprueba siempre el estado del ciclo de vida antes de enviar actualizaciones.
- Elimina automáticamente observadores cuando su ciclo de vida termina, evitando fugas de memoria.
- Facilita la sincronización de datos entre los componentes de la UI y la fuente de datos.

Reloj sin patrones

```
class MainActivity : AppCompatActivity() {  
  
    private lateinit var textViewHora: TextView  
    private var job: Job? = null  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        textViewHora = findViewById(R.id.textView)  
        textViewHora.setOnClickListener {  
            startStopReloj()  
        }  
  
        iniciarReloj()  
    }  
  
    private fun iniciarReloj() {  
        // Cancelamos cualquier job previo  
        job?.cancel()  
        job = lifecycleScope.launch {
```



```
val sdf = SimpleDateFormat("HH:mm:ss", Locale.getDefault())

while (isActive) {
    val millis = System.currentTimeMillis()
    val date = Date(millis)
    textViewHora.text = sdf.format(date)
    delay(1000) // esperar 1 segundo
}

}

fun detenerReloj() {
    job?.cancel()
}

fun startStopReloj() {
    if (job?.isActive == true) {
        detenerReloj()
    } else {
        iniciarReloj()
    }
}

}
```

Reloj con LiveData

Paso 1: Añadir dependencias

implementation ("androidx.lifecycle:lifecycle-viewmodel-ktx:2.9.4")

implementation ("androidx.lifecycle:lifecycle-livedata-ktx:2.9.4")

implementation ("androidx.lifecycle:lifecycle-runtime-ktx:2.9.4")

Paso 2: Configurar layout

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
```



```
<TextView
    android:id="@+id/textView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="0"
    android:textSize="48sp"
    android:textColor="@color/black"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Paso 3: Crear ViewModel

En este ejemplo hemos utilizado Job, que no es mas que handle (controlador) de una coroutine, es decir, representa la ejecución de una coroutine. Nos permite controlarla: cancelarla, comprobar si sigue activa, esperar a que termine, etc.

- job.isActive → true si la coroutine sigue corriendo.
- job.cancel() → cancela la coroutine asociada.

```
class MainViewModel: ViewModel() {
    private val _hora = MutableLiveData<String>()
    private var job: Job? = null

    fun hora(): LiveData<String> {
        return _hora
    }

    fun iniciarReloj() {
        job?.cancel()

        job = viewModelScope.launch {
            val sdf = SimpleDateFormat("HH:mm:ss",
Locale.getDefault())
            while (isActive) {
                val millis = System.currentTimeMillis()
                val date = Date(millis)
                _hora.value = sdf.format(date)

                delay(1000) // esperar 1 segundo
            }
        }
    }
}
```



```
    }  
    }  
}  
  
fun detenerReloj() {  
    job?.cancel()  
}  
  
fun startStopReloj() {  
    if(job?.isActive == true){  
        detenerReloj()  
    }else  
    {  
        iniciarReloj()  
    }  
}  
}
```

Paso 4: Configurar Actividad

```
class MainActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        val viewModel =  
ViewModelProvider(this) [MainViewModel::class.java]  
//val viewModel =  
ViewModelProvider(this).get(MainViewModel::class)  
  
        viewModel.iniciarReloj()  
  
        val textView: TextView = findViewById(R.id.textView)  
  
        viewModel.hora().observe(this) {  
            textView.text = "$it"  
        }  
    }  
}
```



```
    }  
    textView.setOnClickListener {  
        viewModel.startStopReloj()  
    }  
}  
}
```

***** ViewModel NO se destruye al rotar el móvil**



Patrón Flow/StateFlow

Un Flow normal no hace nada hasta que alguien lo colecciona (collect).

StateFlow es un Flow especial que siempre tiene un valor actual

```
class MainViewModel: ViewModel() {
    private val _hora = MutableStateFlow("")
    private var job: Job? = null

    val hora: StateFlow<String> = _hora

    fun iniciarReloj() {
        job?.cancel()

        job = viewModelScope.launch {
            val sdf = SimpleDateFormat("HH:mm:ss",
Locale.getDefault())
            while (isActive) {
                val millis = System.currentTimeMillis()
                val date = Date(millis)
                _hora.value = sdf.format(date)

                delay(1000) // esperar 1 segundo
            }
        }
    }

    fun detenerReloj() {
        job?.cancel()
    }

    fun startStopReloj() {
        if(job?.isActive == true){
            detenerReloj()
        }else
        {
            iniciarReloj()
        }
    }
}
```

```
class MainActivity : AppCompatActivity() {
```



```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    val viewModel =
ViewModelProvider(this) [MainViewModel::class.java]

    viewModel.iniciarReloj()

    val textView: TextView = findViewById(R.id.textView)

    lifecycleScope.launch {
        repeatOnLifecycle(Lifecycle.State.STARTED) {
            viewModel.hora.collect { hora ->
                textView.text = hora
            }
        }
    }

    textView.setOnClickListener {
        viewModel.startStopReloj()
    }
}
```